

CLAUDE CODE: FROM FIRST INSTALL TO LIFE AFTER PRODUCTION

The Token Economy

Claude Code deep dive, episode 10

Clearing is free. Compacting is a large request. Almost nobody knows that.

You asked a one line question at the end of a long day and it drew down your usage like an hour of real work. That is not a bug and it is not unfair billing: your full conversation is sent with every request, so the cost of a question is the length of everything behind it.

This sheet is the habits that follow from that, in the order they are worth adopting. The first one is free and takes a second.

Claude Code changes with every release. Every figure, default and feature name on this sheet was checked on the date below and is stated as of that date. Check the current documentation before you rely on any of it.

In scope

- Why a short question in a long session is not a short request
- What `/usage` actually attributes, and why to run it first
- `/clear` versus `/compact`, and which one costs nothing
- Renaming before clearing, so clearing is filing rather than losing
- Cache lifetime, and the break that quietly reprocesses everything
- The documented ways an idle session spends
- Hooks and skills as context reduction, not just automation

Out of scope

- Plan pricing, which changes and belongs on the vendor's own page
- Any claim about what YOUR bill will be
- Rationing your prompts, which saves almost nothing
- Cost engineering for API products, a different subject entirely



- Published figures, with the qualifiers left attached

The split this sheet runs on: typing less is a junior habit and saves almost nothing. Managing the session is the senior one and is where the money is.

01 The numbers worth carrying

Average per developer, active day	around \$13
Average per developer, month	\$150 to \$250
90% of users stay below	\$30 per active day
Background processes when idle	typically under \$0.04 a session
Agent teams, teammates in plan mode	about 7x a standard session
Cache lifetime, subscription	one hour
Cache lifetime, on usage credits	five minutes
Aim to keep CLAUDE.md under	200 lines

The dollar figures are Anthropic's published averages ACROSS ENTERPRISE DEPLOYMENTS, checked September 2026. They are a sanity check, not a target, and not a prediction about any individual.

03 Why a short question is not a short request

Claude Code sends your full conversation with every request, and each time it uses a tool it sends another request carrying that batch of tool results.

Prompt caching re-reads that history at the cached token rate, which helps enormously and does not make it free. So a one line question in a session that has been open all day still draws usage for the whole conversation.

This is the whole sheet in one idea. Everything below is a way of making sure the thing being carried is worth carrying.

Claude Code sends your full conversation with every request, and each time Claude uses tools it sends another request carrying that batch of tool results.

Claude Code docs, Manage costs effectively, checked September 2026

With prompt caching, Claude Code re-reads that history at the cached token rate, so a one-line question in a session that has been open all day still draws usage for the whole conversation.

Claude Code docs, Manage costs effectively, checked September 2026

04 Run /usage before changing any habit

On a Pro, Max, Team or Enterprise plan, /usage does better than show a number. It ATTRIBUTES recent usage to skills, subagents, plugins and individual MCP servers, each as a percentage of the total.

It flags behaviours such as long context or cache misses when one accounts for 10% or more of recent usage. And it lists your heaviest recent scheduled tasks and loops, ordered by total tokens, with how often each fires and when it last ran.

Press d or w to switch between the last 24 hours and the last 7 days. The figures are computed from local session history on that machine, so other devices and claude.ai are not included.

The reason this is step one rather than step five: the thing consuming your budget is rarely the thing you assumed, and a habit change aimed at the wrong thing costs effort and saves nothing.

On a Pro, Max, Team, or Enterprise plan, /usage shows recent usage attributed to skills, subagents, plugins, and individual MCP servers, each as a percentage of the total.

Claude Code docs, Manage costs effectively, checked September 2026

The breakdown flags behaviors such as long context or cache misses when one accounts for 10% or more of recent usage, and lists the heaviest recent scheduled tasks by total tokens.

Claude Code docs, Manage costs effectively, checked September 2026



05 The asymmetry almost nobody knows

Compacting is not free. `/compact` READS the conversation it summarises, so compacting a large context is itself a large request.

And then the sentence worth memorising, quoted from the documentation: when you want a fresh start instead of continuity, `/clear` costs nothing.

So the rule is simple. Moving to unrelated work? Clear, do not compact out of politeness. Need the history carried forward? Compact, and steer it: `/compact focus on the API changes and the test output`. You can also put a "Compact instructions" section in `CLAUDE.md` so it knows every time.

One thing makes clearing safe rather than frightening: `RENAME FIRST`. `/rename` gives the session a name you will recognise, then `/clear`, then `/resume` if you turn out to have needed it. Renaming before clearing turns throwing away into filing.

```
/compact reads the conversation it summarizes, so compacting a large context is itself a large request.  
When you want a fresh start instead of continuity, /clear costs nothing.
```

```
Claude Code docs, Manage costs effectively, checked September 2026
```

```
Use /clear to start fresh when switching to unrelated work. Use /rename before clearing so you can easily  
find the session later, then /resume to return to it.
```

```
Claude Code docs, Manage costs effectively, checked September 2026
```



06 The cost you cannot see: cache lifetime

Your first message after a break longer than the cache lifetime misses the cache and reprocesses your full context.

The lifetime is ONE HOUR on a subscription, and it drops to FIVE MINUTES once you are drawing on usage credits. On an API key or a cloud provider it is five minutes by default.

Two documented escapes. You can choose the TTL yourself to keep the one hour lifetime while drawing on usage credits. And on Pro and Max plans, resuming a large session after a long break offers to resume from a summary, so later requests do not carry the full history.

A newer Claude Code also prints a Prompt cache line in /usage showing the share of input tokens served from cache, the misses, and whether the cache is warm right now. That turns an invisible cost into a readable one.

The cache lifetime is an hour on a subscription and drops to five minutes once you are drawing on usage credits; on an API key or cloud provider it is five minutes by default.

Claude Code docs, Manage costs effectively, checked September 2026

On Pro and Max plans, resuming a large session after a long break offers to resume from a summary so later requests do not carry the full history.

Claude Code docs, Manage costs effectively, checked September 2026

07 A session can spend while nobody is touching it

Five documented ways, and each of the first three sends your FULL context every time.

A SCHEDULED TASK fires on its interval even while the session is idle. A CROSS-SESSION MESSAGE from another of your sessions is delivered as a new turn when this one sits idle; setting `crossSessionInbound` to hold keeps them queued instead. A GOAL waiting on background work triggers idle check-ins, capped at three per goal between your prompts, and settable to zero.

Then AGENT TEAMMATES, each of which keeps consuming until it exits. And COMPACTION, which as above is a large request in its own right.

Put together: a forgotten loop in a session with a large context is a meter running in an empty room. This is exactly what the Loops rows in `/usage` are for.

A scheduled task fires on its interval even while the session is idle, sending your full context each time.

Claude Code docs, Manage costs effectively, checked September 2026

Claude Code starts at most three idle goal check-ins per goal between your prompts, and each starts a new turn that sends your full context.

Claude Code docs, Manage costs effectively, checked September 2026

Each active agent teammate keeps consuming tokens until it exits.

Claude Code docs, Manage costs effectively, checked September 2026



08 The highest ratio technique here is a grep

Hooks can preprocess data before Claude ever sees it. The documentation's own example: instead of Claude reading a 10,000 line log file to find errors, a PreToolUse hook greps for ERROR and returns only the matching lines, reducing context from tens of thousands of tokens to hundreds.

That is not a small saving, it is a different order of magnitude, and it is a few lines of shell. The shipped example rewrites a test command to "grep -A 5 -E '(FAIL|ERROR|error:)' | head -100" so only failures come back.

The second half of the same idea is skills. CLAUDE.md is loaded into context at session start, so detailed instructions for specific workflows are present even when today's work is unrelated. Skills load on demand. Move the specialised material out and aim to keep CLAUDE.md under 200 lines.

A skill also earns its keep by preventing exploration: a codebase-overview skill describing your architecture and key directories gives Claude that context immediately rather than costing it several file reads to reconstruct.

A hook can grep a 10,000-line log for ERROR and return only matching lines, reducing context from tens of thousands of tokens to hundreds.

Claude Code docs, Manage costs effectively, checked September 2026

CLAUDE.md is loaded into context at session start; skills load on demand. Aim to keep CLAUDE.md under 200 lines by including only essentials.

Claude Code docs, Manage costs effectively, checked September 2026



09 Three more people miss

PREFER CLI TOOLS WHERE ONE EXISTS. MCP tool definitions are deferred by default, so only names and server instructions enter context until a tool is used, but `gh`, `aws`, `gcloud` and `sentry-cli` are still more context efficient because they add no per-tool listing at all. Run `/context` to see what is consuming space, and `/mcp` to disable servers you are not using.

EXTENDED THINKING BILLS AS OUTPUT TOKENS, and the default budget can run to tens of thousands per request depending on the model. For simpler work, lower the effort level with `/effort` or in `/model`. Fable models always use extended thinking and cannot have it turned off.

WRITE SPECIFIC PROMPTS. "Improve this codebase" triggers broad scanning; "add input validation to the login function in `auth.ts`" lets Claude work with minimal file reads. It sounds like productivity advice and is actually cost advice.

And one more worth knowing: delegate verbose operations, such as running tests or processing logs, to subagents, so the bulky output stays in the subagent's context and only a summary returns to your conversation.

CLI tools like `gh`, `aws`, `gcloud` and `sentry-cli` are more context-efficient than MCP servers because they do not add any per-tool listing.

Claude Code docs, Manage costs effectively, checked September 2026

Thinking tokens are billed as output tokens, and the default budget can be tens of thousands of tokens per request depending on the model.

Claude Code docs, Manage costs effectively, checked September 2026

Delegate verbose operations to subagents so the output stays in the subagent context while only a summary returns to the main conversation.

Claude Code docs, Manage costs effectively, checked September 2026

10 The published figures, with their qualifiers

Across enterprise deployments, the published average is around \$13 per developer per active day and \$150 to \$250 per developer per month, with costs remaining below \$30 per active day for 90% of users.

Read the qualifier, because it is the point: that is an average across ENTERPRISE DEPLOYMENTS, checked September 2026. It is not a prediction about you, and per-developer costs vary widely with model selection, codebase size and how many instances are running.

Two smaller figures worth carrying. Background processes consume a small amount even when idle, typically under \$0.04 per session. And agent teams use approximately 7x more tokens than standard sessions when teammates run in plan mode, because each teammate maintains its own context window.

The honest use of all of this is to notice when you are an order of magnitude away from it, not to aim at it.

Across enterprise deployments, the average cost is around \$13 per developer per active day and \$150-250 per developer per month, with costs remaining below \$30 per active day for 90% of users.

Claude Code docs, Manage costs effectively, checked September 2026

Background processes consume a small amount of tokens, typically under \$0.04 per session, even without active interaction.

Claude Code docs, Manage costs effectively, checked September 2026

Agent teams use approximately 7x more tokens than standard sessions when teammates run in plan mode.

Claude Code docs, Manage costs effectively, checked September 2026

The token economy, as a checklist

1. Have you run `/usage` this week, or are you guessing?
2. Is this session still about the thing it started as? If not, `/rename` then `/clear`
3. Do you actually need continuity here? If not, do not compact, clear
4. When you do compact, did you tell it what to keep?
5. About to leave a large session for an hour? Clear it rather than let it go cold
6. Any scheduled task or loop still running that you have forgotten about?
7. Any agent teammate still alive whose work is finished?
8. Is there a verbose command whose output a hook could filter first?
9. Is `CLAUDE.md` under 200 lines, with the specialised parts moved to skills?
10. Is this prompt specific enough to avoid a repository-wide scan?

Everything else

Site

<https://code-with-sam-dev.github.io>

YouTube

<https://www.youtube.com/@CodewithSam-Dev>

LinkedIn

<https://www.linkedin.com/company/code-with-sam-dev>

X

<https://x.com/CodeWithSamDev>

TikTok

<https://www.tiktok.com/@codewithsamdev>

Instagram

<https://www.instagram.com/codewithsamdev>

GitHub

<https://github.com/code-with-sam-dev>

Manage costs effectively

<https://code.claude.com/docs/en/costs>

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Claude and Claude Code are products of Anthropic. This is an independent, unofficial guide produced by Code with Sam. It is not affiliated with, endorsed by, or sponsored by Anthropic, and no Anthropic artwork is reproduced here. Product names appear only to identify the software being discussed. Other names are the property of their respective owners.

Technical claims verified 2026-09-13 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.